

透過 Media CDN 和 Live Streaming API 在 Google Cloud 直播

來源：<https://codelabs.developers.google.com/mediacdnlive-codelab?hl=zh-tw>

步驟數：18

本研究室將逐步引導您使用 Media CDN (CDN) 和 #43，部署即時串流工作流程示範。Live Stream API #43；Cloud Storage #43；媒體播放器。

目錄

1. 簡介
2. 事前準備
3. 設定和需求
4. Google Cloud SDK 版本
5. 必要條件
6. 建立 Cloud Storage bucket
7. 設定 Live Streaming API 環境
8. 建立及設定輸入端點
9. 建立及設定直播頻道
10. 啟動 Live Stream 頻道
11. 設定 Media CDN
12. 產生輸入信號
13. 開啟新的 Cloud Shell
14. 確認直播訊號是否正在轉碼
15. 確認 Edge Cache Service 執行個體是否正常運作
16. 使用 VLC 串流播放轉碼後的即時訊號影片
17. 監控 Media CDN
18. 清理實驗室環境

1. 簡介

內容傳遞網路 (CDN) 會將經常存取的內容快取在離使用者較近的位置，終止與用戶端較近的連線，重複使用與來源的連線，並採用現代網路通訊協定和自訂項目，藉此提升使用者效能。對使用者 (和我們的客戶) 而言，這代表延遲時間縮短、可靠性提高，以及成本降低，進而提升銷售業績、網頁瀏覽體驗和使用者體驗。如今，很少有現代網站和影片串流平台不使用 CDN。

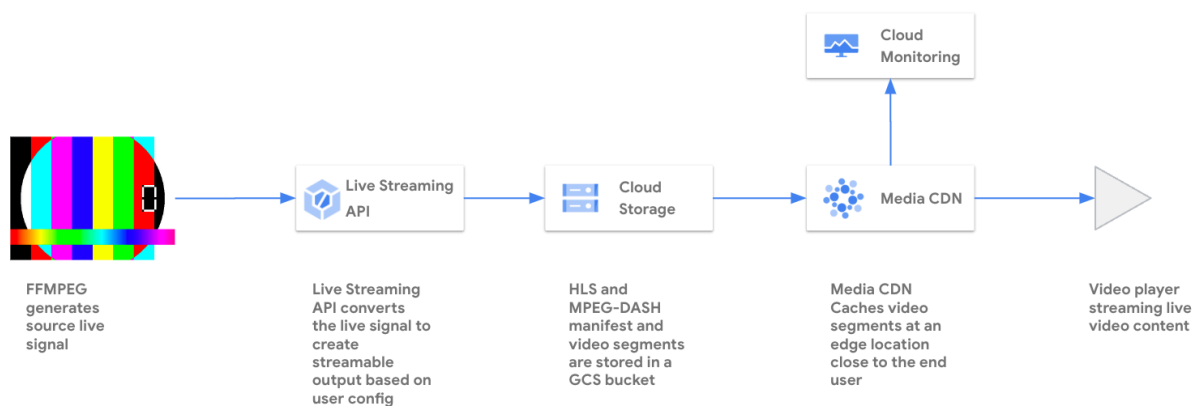
課程內容

本實驗室將引導我們逐步部署直播工作流程環境，其中包含 **Media CDN (CDN) + Cloud Media Live Streaming API** (即時影像轉碼) + **Cloud Storage** (影片儲存空間) + **Video Player** (VLC、Google Shaka Player 等，支援 HLS + MPEG-DASH 的播放器)。

我們會設定 Live Stream API 元件 (輸入內容和管道)，並使用 FFmpeg 將現行動態饋給傳送至輸入內容/管道 (FFmpeg 可以產生即時測試信號)。Live Streaming API 會將現行動態饋給轉碼。轉碼後的影片資訊清單和片段會儲存在 Cloud Storage bucket 中。接著，我們會將即時影像 Cloud Storage bucket 設為來源，並設定 Media CDN。最後，VLC Player 會用於播放透過 Media CDN 快取的即時內容。我們也會設定 Cloud Monitoring 資訊主頁，以圖表呈現 Media CDN 的活動。

建構項目

在本實驗室中，我們將根據下列架構設定環境：



在本實驗室中，我們會設定下列元件並執行下列工作：

- 建立 Google Cloud Storage (GCS) bucket，用於儲存即時轉碼影片
- 設定 Live Streaming API，將影片轉碼為多種格式：HTTP 即時串流 + MPEG DASH、SD 和 HD
- 設定直播元件：輸入/頻道
- 啟動 Live Stream 頻道
- 設定 Media CDN，並以 GCS bucket 做為來源
- 設定 FFmpeg，提供直播頻道內容
- 使用影片播放器串流播放轉碼後的即時影像
- 設定 Cloud Monitoring 資訊主頁，監控 Media CDN 活動 (延遲時間、快取命中、快取失敗等)

注意：在本實驗室中，我們假設使用者可以存取 [Google Cloud 控制台](#)，且已設定專案。我們也假設使用者從全新環境開始，且未為這個示範設定任何項目。

所有設定操作都會透過 [Cloud Shell](#) 的指令列完成。我們隨時可以在控制台中，查看透過指令列設定的元件。實驗室中會出現指標，指向 Google Cloud 控制台。

注意：Media CDN 存取權受到限制。如要存取 Media CDN，請與帳戶團隊聯絡。他們可以代您提出存取要求。如果您是 Google 員工，並想使用 Media CDN 測試 Live Streaming，請與 Media CDN 的 PM 聯絡，要求存取 Media CDN。

步驟 2 · 約 3 分鐘

2. 事前準備

媒體 CDN 存取權受到限制。如要存取 Media CDN，請與帳戶團隊聯絡。他們可以代您提出存取要求。如果您是 Google 員工，並想使用 Media CDN 測試 Live Streaming，請與 Media CDN 的 PM 聯絡，要求存取 Media CDN。

注意：請確認您的專案已加入 Media CDN 允許清單。如果專案未加入許可清單，您就無法完成本實驗室。您可能會在 GCP 主控台中看到 Media CDN 服務，但無法建立來源或服務。

步驟 3 · 約 3 分鐘

3. 設定和需求

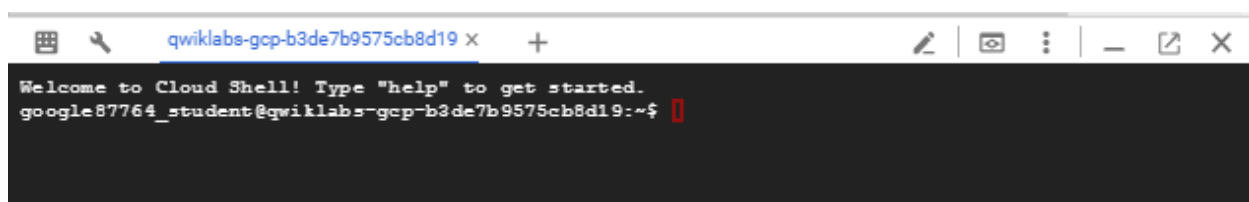
啟動 Cloud Shell

雖然可以透過筆電遠端操作 Google Cloud，但在本程式碼研究室中，您將使用 [Google Cloud Shell](#)，這是可在雲端執行的指令列環境。

在 [Google Cloud 控制台](#) 中，點選右上工具列的 Cloud Shell 圖示：



佈建並連線至環境的作業需要一些時間才能完成。完成後，您應該會看到如下的內容：

A screenshot of a web browser window showing the Google Cloud Shell terminal. The browser's address bar displays 'qwiklabs-gcp-b3de7b9575cb8d19'. The terminal window has a dark background and shows the following text: 'Welcome to Cloud Shell! Type "help" to get started.' followed by the prompt 'google87764_student@qwiklabs-gcp-b3de7b9575cb8d19:~\$' and a red cursor. The terminal window includes standard browser controls like back, forward, and search buttons.

這部虛擬機器搭載各種您需要的開發工具，並提供永久的 5GB 主目錄，而且可在 Google Cloud 運作，大幅提升網路效能並強化驗證功能。您可以在瀏覽器中完成本程式碼研究室的所有作業。您不需要安裝任何軟體。

步驟 4 · 約 5 分鐘

4. Google Cloud SDK 版本

截至本文撰寫時間為止，`408.0.0` 是最新版的 Google Cloud SDK。本實驗室中的所有指令都經過測試，可搭配最新版 Google Cloud SDK 使用。繼續操作前，請確認 Cloud Shell 使用的是最新版 SDK。

檢查 SDK 版本

我們會使用 `gcloud version` 指令檢查 SDK 版本。

指令

```
gcloud version | grep "Google Cloud SDK"
```

輸出內容範例

```
Google Cloud SDK 408.0.0
```

後續步驟

注意：請根據 `gcloud version` 指令的輸出內容選取選項 1 或 2

1. 如果 SDK 版本為 `408.0.0` 以上，請跳至下一節。
2. 如果 SDK 版本低於 `408.0.0`，請執行下列指令更新 SDK。

```
sudo apt-get update && sudo apt-get install google-cloud-sdk
```

5. 必要條件

開始設定 GCP 資源前，請先完成下列步驟：

1. 設定環境變數
2. 啟用必要的服務 API

1. 設定環境變數

在本實驗室中，我們會使用幾個變數執行 `gcloud` 和 `curl` 指令。我們需要設定下列環境變數。

- 專案 ID
- 專案編號
- 使用者名稱
- 區域
- 輸入 ID
- 頻道 ID

專案 ID 和使用者名稱

這些環境變數通常會在 Cloud Shell 中預先設定。我們會使用 `env` 指令進行驗證。

指令

```
env | grep -E 'DEVSHELL_PROJECT_ID=|LOGNAME'
```

輸出內容範例

```
DEVSHELL_PROJECT_ID=<YOUR_PROJECT_ID>  
LOGNAME=<YOUR_USERNAME>
```

建立 `env_variables` 檔案

使用 `cat` 指令建立 `env_variables.txt` 檔案。下列指令會在使用者主目錄中建立 `env_variables.txt` 檔案。

注意：如果系統提示，請授權要求。

指令

```
cat > ~/env_variables.txt << EOF  
export PROJECT_NUMBER=$(gcloud projects describe $GOOGLE_CLOUD_PROJECT --  
format="value(projectNumber)")  
export LOCATION=us-west2  
export INPUT_ID=lab-live-input  
export CHANNEL_ID=lab-live-channel  
EOF
```


設定環境變數

我們將使用 `source` 指令設定環境變數

指令

```
source ~/env_variables.txt
```

注意：上述指令只會為 Cloud Shell 執行個體生命週期設定環境變數。如果使用者離開 Cloud Shell 執行個體或開啟新的 Cloud Shell，則必須重新執行 `source` 指令。

確認已設定變數

請確認所有必要環境變數都已設定。輸出內容應會顯示總共 6 個環境變數。

指令

```
env | grep -E  
'DEVSHELL_PROJECT_ID=|LOGNAME|PROJECT_NUMBER|LOCATION|INPUT_ID|CHANNEL_ID'
```

輸出內容範例

```
LOCATION=us-west2  
DEVSHELL_PROJECT_ID=<YOUR_PROJECT_ID>  
LOGNAME=<YOUR_USERNAME>  
PROJECT_NUMBER=<YOUR_PROJECT_NUMBER>  
INPUT_ID=lab-live-input  
CHANNEL_ID=lab-live-channel
```

2. 啟用必要的服務 API

我們需要確保專案已啟用下列 API。

- Network Services API
- Certificate Manager API
- Livestream API
- Media CDN Edge Cache API

啟用 Network Services API

如要啟用 Network Services API，請執行下列指令：

指令

```
gcloud services enable networkservices.googleapis.com
```

啟用 Certificate Manager API

如要啟用 Certificate Manager API，請執行下列指令：

指令

```
gcloud services enable certificatemanager.googleapis.com
```

啟用 Live Stream API

如要啟用 Live Stream API，請執行下列指令：

指令

```
gcloud services enable livestream.googleapis.com
```

啟用 Media CDN Edge Cache API

如要啟用 Media CDN Edge Cache API，請執行下列指令：

指令

```
gcloud services enable edgecache.googleapis.com
```

確認 API 已啟用

執行 `gcloud services list` 指令，列出所有已啟用的 API。輸出內容應會顯示 4 個 API。

指令

```
gcloud services list | grep -E  
'networkservices|certificatemanager|livestream|edgecache'
```

輸出內容範例

```
NAME: certificatemanager.googleapis.com  
NAME: livestream.googleapis.com  
NAME: networkservices.googleapis.com  
NAME: edgecache.googleapis.com
```

6. 建立 Cloud Storage bucket

注意：Cloud Storage bucket 名稱在全域範圍內都不可重複。為確保沒有衝突，我們會使用 \$LOGNAME 環境變數做為值區名稱的一部分。系統預設會在 Cloud Shell 中建立這個變數。

在本節中，我們將執行下列操作：

1. 建立 Cloud Storage 值區
2. 將 bucket 設為公開存取

稍後在本實驗室中，我們會使用這個 bucket 儲存轉碼後的影片檔。這個 bucket 也會做為 Media CDN 服務的來源儲存空間。

1. 建立 bucket

我們會使用 `gsutil mb` 指令建立 bucket：

指令

```
gsutil mb gs://live-streaming-storage-$LOGNAME
```

2. 將值區設為公開存取

注意：為求簡單，我們將來源 bucket 設為公開可讀取。Media CDN 也支援將私人值區做為來源。如要進一步瞭解如何搭配 Media CDN 使用私有 Cloud Storage bucket，請參閱[公開說明文件](#)。

我們將使用 `gsutil iam` 指令公開檔案：

指令

```
gsutil iam ch allUsers:objectViewer gs://live-streaming-storage-$LOGNAME
```

步驟 7 · 約 3 分鐘

7. 設定 Live Streaming API 環境

Live Streaming API 鏈的元件架構如下：



我們在上一節中建立了 Cloud Storage bucket `live-streaming-storage-$LOGNAME`。在接下來兩節中，我們將建立下列資源：

- **直播輸入：**輸入端點是編碼器傳送輸入串流的端點。您可以使用輸入端點指定串流的設定，例如輸入解析度、輸入類型和影片裁剪。
- **直播頻道：**頻道是一種資源，可透過輸入端點攝取輸入串流、將輸入串流轉碼為多種轉譯版本，並在指定位置以特定格式發布輸出直播串流。您可以在同一個頻道中加入主要和備用輸入串流。

我們稍後會在實驗室中建立下列資源：

- **編碼器：**編碼器是用來傳送輸入串流的程式。在本實驗室中，我們將使用 FFmpeg。
-

8. 建立及設定輸入端點

建立 input.json 檔案

我們會建立 `input.json` 檔案，指定直播訊號類型。在本實驗室中，我們將使用 RTMP 直播信號。

指令

```
cat > input.json << EOF
{
  "type": "RTMP_PUSH"
}
EOF
```

建立輸入端點

撰寫本實驗室內容時，gcloud 尚不支援 Live Stream API。我們將使用 `curl` 指令發出 API 呼叫。

注意：下列指令會從先前設定的環境變數中，選取 `PROJECT_NUMBER`、`LOCATION` 和 `INPUT_ID`。

指令

```
curl -X POST \
-H "Authorization: Bearer "$(gcloud auth application-default print-access-token) \
-H "Content-Type: application/json; charset=utf-8" \
-d @input.json \
"https://livestream.googleapis.com/v1/projects/$PROJECT_NUMBER/locations/$LOCATION/inputs?inputId=$INPUT_ID"
```

輸出內容範例

```
{
  "name": "projects/PROJECT_NUMBER/locations/us-west2/operations/operation-1661405972853-5e70a38d6f27f-79100d00-310671b4",
  "metadata": {
    "@type": "type.googleapis.com/google.cloud.video.livestream.v1.OperationMetadata",
    "createTime": "2022-08-25T05:39:32.884030164Z",
    "target": "projects/PROJECT_NUMBER/locations/us-west2/inputs/lab-live-input",
    "verb": "create",
    "requestedCancellation": false,
    "apiVersion": "v1"
  },
  "done": false
}
```

輸出內容包含許多實用資訊，但目前我們需要著重於兩個欄位：

- **作業 ID**：從輸出內容複製並記下作業 ID。以下是輸出範例中的作業 ID。這項資訊位於以 `"name"` 開頭的輸出行。`"operation-1661405972853-5e70a38d6f27f-79100d00-310671b4"`
- **狀態**：我們需要等待狀態從 `"done": false` 變更為 `"done": true`

檢查狀態

繼續下一步之前，請先確認輸入端點已成功建立並準備就緒。

注意：如果回應包含 `"done: false"`，請重複執行指令，直到回應包含 `"done: true"` 為止。在區域中建立第一個輸入端點最多可能需要 10 分鐘。

在下方指令中，將 `<OPERATION>` 替換為我們剛才取得的作業 ID。在本例中為 `"operation-1661405972853-5e70a38d6f27f-79100d00-310671b4"`。

注意：下列指令只會為 Cloud Shell 執行個體生命週期設定環境變數。如果使用者離開 Cloud Shell 執行個體或開啟新的 Cloud Shell，則需要再次設定環境變數。

指令

```
export OPERATION_ID_1=<OPERATION>
```

注意：您可能需要多次執行該指令，直到輸出內容中顯示 `"done:true"`，表示輸入端點已建立完成並可供使用。

指令

```
curl -X GET \  
-H "Authorization: Bearer "$(gcloud auth application-default print-access-token) \  
"https://livestream.googleapis.com/v1/projects/$PROJECT_NUMBER/locations/$LOCATION/op  
erations/$OPERATION_ID_1"
```

輸出內容範例

```
{
  "name": "projects/PROJECT_NUMBER/locations/us-west2/operations/operation-1661408816982-5e70ae25cea49-617844f0-8fdcb4a1",
  "metadata": {
    "@type":
"type.googleapis.com/google.cloud.video.livestream.v1.OperationMetadata",
    "createTime": "2022-08-25T06:26:57.001530499Z",
    "endTime": "2022-08-25T06:26:57.043623522Z",
    "target": "projects/PROJECT_NUMBER/locations/us-west2/inputs/lab-live-input",
    "verb": "create",
    "requestedCancellation": false,
    "apiVersion": "v1"
  },
  "done": true,
  "response": {
    "@type": "type.googleapis.com/google.cloud.video.livestream.v1.Input",
    "name": "projects/PROJECT_ID/locations/us-west2/inputs/lab-live-input",
    "createTime": "2022-08-25T06:26:56.997623672Z",
    "updateTime": "2022-08-25T06:26:56.997623672Z",
    "type": "RTMP_PUSH",
    "uri": "rtmp://34.94.97.220/live/4b7846a1-4a67-44ed-bfd0-d98281b6464a",
    "tier": "HD"
  }
}
```

重複執行指令，直到看到 `"done:true"`，表示輸入端點已建立並準備就緒。

注意：「uri」 - 在這裡 `rtmp://34.94.97.220/live/4b7846a1-4a67-44ed-bfd0-d98281b6464a` 複製並記下 URI，我們會在實驗室的後續步驟中使用這個 URI，將即時信號饋送至 Input/Channel。

儲存 URI

我們稍後會在實驗室中使用先前的輸出內容 `URI`。現在，請為 `URI` 設定環境變數。

注意：下列指令只會為 Cloud Shell 執行個體生命週期設定環境變數。如果使用者離開 Cloud Shell 執行個體或開啟新的 Cloud Shell，則需要再次設定環境變數。

指令

```
export URI=<uri>
```

將 `<uri>` 換成您稍早記下的 URI。您也可以視需要使用 GET 方法擷取 URI

指令

```
curl -s -X GET -H "Authorization: Bearer "$(gcloud auth application-default print-  
access-token)  
"https://livestream.googleapis.com/v1/projects/$PROJECT_NUMBER/locations/$LOCATION/in  
puts/$INPUT_ID" | jq .uri
```

步驟 9 · 約 15 分鐘

9. 建立及設定直播頻道

現在來建立直播頻道，並與我們在上一個部分建立的輸入端點建立關聯。以下範例會建立頻道，產生由單一高畫質 (1280x720) 轉譯內容組成的 HLS 直播串流。管道會與輸入端點和先前建立的儲存空間 bucket 建立關聯。

建立 channel.json 檔案

在 Cloud Shell 終端機中輸入下列指令，建立 `"channel.json"` 檔案：

指令

```
cat > channel.json << EOF
{
  "inputAttachments": [
    {
      "key": "my-input",
      "input": "projects/$PROJECT_NUMBER/locations/$LOCATION/inputs/$INPUT_ID"
    }
  ],
  "output": {
    "uri": "gs://live-streaming-storage-$LOGNAME"
  },
  "elementaryStreams": [
    {
      "key": "es_video",
      "videoStream": {
        "h264": {
          "profile": "high",
          "widthPixels": 1280,
          "heightPixels": 720,
          "bitrateBps": 3000000,
          "frameRate": 30
        }
      }
    },
    {
      "key": "es_audio",
      "audioStream": {
        "codec": "aac",
        "channelCount": 2,
        "bitrateBps": 160000
      }
    }
  ],
  "muxStreams": [
    {
      "key": "mux_video_ts",
      "container": "ts",
      "elementaryStreams": ["es_video", "es_audio"],
      "segmentSettings": { "segmentDuration": "2s" }
    }
  ],
  "manifests": [
    {
      "fileName": "main.m3u8",
      "type": "HLS",
      "muxStreams": [
        "mux_video_ts"
      ]
    }
  ],
}
```

```
    "maxSegmentCount": 5
  }
]
}
EOF
```

建立頻道

執行下列 `curl` 指令來建立管道：

指令

```
curl -X POST \
-H "Authorization: Bearer "$(gcloud auth application-default print-access-token) \
-H "Content-Type: application/json; charset=utf-8" \
-d @channel.json \
"https://livestream.googleapis.com/v1/projects/$PROJECT_NUMBER/locations/$LOCATION/channels?channelId=$CHANNEL_ID"
```

輸出內容範例

```
{
  "name": "projects/PROJECT_NUMBER/locations/us-west2/operations/operation-1661405972853-5e70a38d6f27f-79100d00-310671b4",
  "metadata": {
    "@type":
"type.googleapis.com/google.cloud.video.livestream.v1.OperationMetadata",
    "createTime": "2022-08-25T05:39:32.884030164Z",
    "target": "projects/PROJECT_NUMBER/locations/us-west2/channels/lab-live-channel",
    "verb": "create",
    "requestedCancellation": false,
    "apiVersion": "v1"
  },
  "done": false
}
```

記下並複製作業 ID。我們會在接下來的步驟中用到這項資訊。您可以在以 `"name"` 開頭的輸出行中找到這項資訊。

檢查狀態

在繼續處理之前，我們需要確認頻道已成功建立並準備就緒。

注意：如果回應包含 `"done: false"`，請重複執行指令，直到回應包含 `"done: true"` 為止。在區域中建立第一個頻道最多可能需要 10 分鐘。

在下方指令中，將 `<OPERATION>` 替換為我們剛才取得的作業 ID。在本範例中，這個屬性的值是 `operation-1661405972853-5e70a38d6f27f-79100d00-310671b4`。

指令

```
export OPERATION_ID_2=<OPERATION>
```

注意：您可能需要多次執行該指令，直到輸出內容中顯示「done:true」，表示輸入端點已建立完成並可供使用。

指令

```
curl -s -X GET \  
-H "Authorization: Bearer "$(gcloud auth application-default print-access-token) \  
"https://livestream.googleapis.com/v1/projects/$PROJECT_NUMBER/locations/$LOCATION/op  
erations/$OPERATION_ID_2"
```

輸出內容範例

```
"name": "projects/PROJECT_NUMBER/locations/us-west2/operations/operation-1668666801461-5eda4c3f31852-a4d2229f-0efeef9e",
"metadata": {
  "@type":
"type.googleapis.com/google.cloud.video.livestream.v1.OperationMetadata",
  "createTime": "2022-11-17T06:33:21.500841488Z",
  "endTime": "2022-11-17T06:33:21.529311112Z",
  "target": "projects/PROJECT_NUMBER/locations/us-west2/channels/lab-live-channel",
  "verb": "create",
  "requestedCancellation": false,
  "apiVersion": "v1"
},
"done": true,
"response": {
  "@type": "type.googleapis.com/google.cloud.video.livestream.v1.Channel",
  "name": "projects/PROJECT_NAME/locations/us-west2/channels/lab-live-channel",
  "createTime": "2022-11-17T06:33:21.497818033Z",
  "updateTime": "2022-11-17T06:33:21.497818033Z",
  "activeInput": "my-input",
  "output": {
    "uri": "gs://live-streaming-storage-LOGNAME"
  },
  "elementaryStreams": [
    {
      "videoStream": {
        "h264": {
          "widthPixels": 1280,
          "heightPixels": 720,
          "frameRate": 30,
          "bitrateBps": 3000000,
          "gopDuration": "2s",
          "vbvSizeBits": 3000000,
          "vbvFullnessBits": 2700000,
          "entropyCoder": "cabac",
          "profile": "high"
        }
      },
      "key": "es_video"
    },
    {
      "audioStream": {
        "codec": "aac",
        "bitrateBps": 160000,
        "channelCount": 2,
        "sampleRateHertz": 48000
      },
      "key": "es_audio"
    }
  ]
}
```

```

],
"muxStreams": [
  {
    "key": "mux_video_ts",
    "container": "ts",
    "elementaryStreams": [
      "es_video",
      "es_audio"
    ],
    "segmentSettings": {
      "segmentDuration": "2s"
    }
  }
],
"manifests": [
  {
    "fileName": "main.m3u8",
    "type": "HLS",
    "muxStreams": [
      "mux_video_ts"
    ],
    "maxSegmentCount": 5,
    "segmentKeepDuration": "60s"
  }
],
"streamingState": "STOPPED",
"inputAttachments": [
  {
    "key": "my-input",
    "input": "projects/PROJECT_NUMBER/locations/us-west2/inputs/lab-live-input"
  }
],
"logConfig": {
  "logSeverity": "OFF"
}
}
}

```

重複執行指令，直到看到 `"done:true"`，表示輸入端點已建立並準備就緒。

請注意，目前 `"streamingState"` 為 `"STOPPED"`；我們將在下一節啟動管道。

10. 啟動 Live Stream 頻道

建立直播頻道後，接著啟動頻道。在本節中，我們將執行下列操作：

1. 啟動 Live Streaming 頻道
2. 檢查頻道狀態，確認 `streamingState` 為 `"AWAITING INPUT"`

1. 啟動頻道

在 Cloud Shell 中執行下列 `curl` 指令，啟動管道：

指令

```
curl -X POST \  
-H "Authorization: Bearer "$(gcloud auth application-default print-access-token) \  
-H "Content-Type: application/json; charset=utf-8" \  
-d "" \  
"https://livestream.googleapis.com/v1/projects/$PROJECT_NUMBER/locations/$LOCATION/channels/$CHANNEL_ID:start"
```

輸出內容範例

```
{  
  "name": "projects/PROJECT_NUMBER/locations/LOCATION/operations/operation-1661405972853-5e70a38d6f27f-79100d00-310671b4",  
  "metadata": {  
    "@type":  
"type.googleapis.com/google.cloud.video.livestream.v1.OperationMetadata",  
    "createTime": "2022-08-25T05:39:32.884030164Z",  
    "target": "projects/PROJECT_NUMBER/locations/us-west2/channels/lab-live-channel",  
    "verb": "start",  
    "requestedCancellation": false,  
    "apiVersion": "v1"  
  },  
  "done": false  
}
```

注意：在區域中啟動第一個頻道約需 10 分鐘。如要驗證管道狀態，請繼續實驗室的操作，並按照下一節的指示執行。看到 `"streamingState": "AWAITING INPUT"` 時再按「下一步」，表示頻道已啟動，可以接收直播訊號進行轉碼。

2. 檢查頻道狀態

執行下列 `curl` 指令，取得 Channel 的狀態：

注意：此時請再次等待，然後重新執行 GET 指令，直到看到 `"streamingState": "AWAITING_INPUT"` 為止，表示頻道已啟動並準備接收輸入訊號。

指令

```
curl -s -X GET -H "Authorization: Bearer "$(gcloud auth application-default print-access-token)
"https://livestream.googleapis.com/v1/projects/$PROJECT_NUMBER/locations/$LOCATION/channels/$CHANNEL_ID" | grep "streamingState"
```

輸出內容範例

```
"streamingState": "AWAITING_INPUT",
```

重複執行指令，直到看到「`AWAITING_INPUT`」，表示頻道正在執行，且已準備好接收訊號。

11. 設定 Media CDN

在本節中，我們將部署 CDN 基礎架構 Media CDN。我們會建立下列資源：

1. 邊緣快取來源
2. 邊緣快取服務

警告：請勿在網址路徑或查詢參數中加入機密、私密或個人識別資訊。根據《[服務專屬條款](#)》第 12 節 (資源欄位)，網址 (包括主機、路徑和查詢參數) 會視為無法識別的服務資料。根據隱私權和安全性最佳做法，網址元件應是不透明的系統產生 ID，而非衍生自使用者輸入內容。

1. 建立邊緣快取來源

邊緣快取來源代表內容位置，例如 Cloud Storage bucket、第三方儲存位置或負載平衡器。以 CDN 來說，原始伺服器是指要發布內容的來源所在位置，例如所有 CSS、JavaScript、HTML、圖片等。在本實驗室中，我們將建立對應至實驗室開頭建立的 Cloud Storage bucket 的原始伺服器。我們將邊緣快取來源稱為 `cme-origin`。CDN 來源是指所有來源內容的儲存位置，這些內容會先儲存在這裡，再發布至邊緣快取伺服器。

我們會使用 `gcloud edge-cache origins create` 指令建立來源。指令會在幾分鐘內執行完畢。

注意：Media CDN 最多需要 10 分鐘才能建立來源。

指令

```
gcloud edge-cache origins create cme-origin \
--origin-address="gs://live-streaming-storage-$LOGNAME"
```

輸出內容示例

```
Create request issued for: cme-origin
Waiting for operation [projects/my-project/locations/global/operations/operation-
1612121774168-5ba3759af1919-
3fdcd7b1-99f59223] to complete...done
Created origin cme-origin
```

2. 建立邊緣快取服務

現在我們已設定邊緣快取來源，可以建立邊緣快取服務本身。

建立 `cme-demo.yaml` 檔案

邊緣快取服務設定是透過 `YAML` 檔案完成。在 Cloud Shell 中，建立名為 `cme-demo.yaml` 的本機檔案。使用 `vi`、`nano` 或任何其他編輯器，並在 `YAML` 檔案中貼上下列程式碼：

```
name: cme-demo
routing:
  hostRules:
    - hosts:
        - demo.cme.com
      pathMatcher: routes
  pathMatchers:
    - name: routes
      routeRules:
        - headerAction:
            responseHeadersToAdd:
              - headerName: x-cache-status
                headerValue: "{cdn_cache_status}"
          matchRules:
            - prefixMatch: /
          origin: cme-origin
          priority: 100
          routeAction:
            cdnPolicy:
              cacheKeyPolicy: {}
              cacheMode: FORCE_CACHE_ALL
              defaultTtl: 3600s
              signedRequestMode: DISABLED
        - headerAction:
            responseHeadersToAdd:
              - headerName: x-cache-status
                headerValue: "{cdn_cache_status}"
          matchRules:
            - pathTemplateMatch: /**.m3u8
          origin: cme-origin
          priority: 25
          routeAction:
            cdnPolicy:
              cacheKeyPolicy: {}
              cacheMode: FORCE_CACHE_ALL
              defaultTtl: 1s
              signedRequestMode: DISABLED
        - headerAction: {}
          matchRules:
            - pathTemplateMatch: /**.ts
          origin: cme-origin
          priority: 50
          routeAction:
            cdnPolicy:
              cacheKeyPolicy: {}
              cacheMode: FORCE_CACHE_ALL
```

```
defaultTtl: 2s
signedRequestMode: DISABLED
```

我們將保留所有邊緣快取服務的預設設定。在上述檔案中，使用者可能想更新 3 個欄位值：

- `name`：Media CDN 執行個體的名稱，在此為 `cme-demo`
- `hosts`：這項 Media CDN 服務將解析的網域名稱清單 - 位於：`demo.cme.com`。我們會在示範期間使用這項功能。我們會使用 Media CDN 執行個體的 IP 位址。
- `origin`：這是我們在上一個步驟中建立的 Edge Cache Origin。設為 `cme-origin` - Media CDN 來源的名稱。

注意：串流工作階段開始時，用戶端會先下載 m3u8 檔案。m3u8 檔案包含影片片段檔案 (.ts 檔案) 的相關資訊。在直播使用案例中，轉碼後的檔案會持續上傳至儲存空間 bucket，因此 m3u8 檔案會頻繁更新。為確保直播正常運作，我們希望 Media CDN 只會短時間快取 m3u8 檔案。

在上述設定檔中，我們為 .m3u8 檔案建立了個別的路徑規則，TTL 則設為 1 秒

如要進一步瞭解可在 YAML 檔案中使用的不同變數，請參閱 [Edge Cache Service 設定指南](#)。

建立邊緣快取服務

我們將在邊緣快取來源 `cme-origin` 上，建立名為 `cme-demo` 的邊緣快取服務，主機為 `demo.cme.com`。如要建立服務，請在 Cloud Shell 中執行下列指令：

注意：Media CDN 最多可能需要 10 分鐘才能建立服務

指令

```
gcloud edge-cache services import cme-demo \
  --source=cme-demo.yaml
```

建立 Edge Cache Service 可能需要幾分鐘的時間。

輸出內容範例

```
Request issued for: [cme-demo]
Waiting for operation [projects/PROJECT_ID/locations/global/operations/operation-1670476252264-5ef4a0f9f36ce-dd380af5-321be9a0] to complete...done.
createTime: '2022-12-07T18:08:54.403446942Z'
ipv4Addresses:
- 34.104.35.152
ipv6Addresses:
- '2600:1900:4110:d18::'
name: projects/PROJECT_ID/locations/global/edgeCacheServices/cme-demo
routing:
  hostRules:
  - hosts:
    - demo.cme.com
    - 34.104.35.152
    pathMatcher: routes
  pathMatchers:
  - name: routes
    routeRules:
    - headerAction:
      responseHeadersToAdd:
      - headerName: x-cache-status
        headerValue: '{cdn_cache_status}'
    matchRules:
    - prefixMatch: /
    origin: projects/123456789/locations/global/edgeCacheOrigins/cme-origin
    priority: '100'
    routeAction:
      cdnPolicy:
        cacheKeyPolicy: {}
        cacheMode: FORCE_CACHE_ALL
        defaultTtl: 3600s
        signedRequestMode: DISABLED
    - headerAction:
      responseHeadersToAdd:
      - headerName: x-cache-status
        headerValue: '{cdn_cache_status}'
    matchRules:
    - pathTemplateMatch: /**.m3u8
    origin: projects/123456789/locations/global/edgeCacheOrigins/cme-origin
    priority: '25'
    routeAction:
      cdnPolicy:
        cacheKeyPolicy: {}
        cacheMode: FORCE_CACHE_ALL
        defaultTtl: 1s
        signedRequestMode: DISABLED
    - headerAction: {}
    matchRules:
```

```
- pathTemplateMatch: /**.ts
origin: projects/123456789/locations/global/edgeCacheOrigins/cme-origin
priority: '50'
routeAction:
  cdnPolicy:
    cacheKeyPolicy: {}
    cacheMode: FORCE_CACHE_ALL
    defaultTtl: 2s
    signedRequestMode: DISABLED
updateTime: '2022-12-08T05:11:31.598744308Z'
```

記下並複製 Edge Cache Service 執行個體的 `ipv4Addresses` - 這裡 `34.104.36.157`。我們會使用這個檔案更新 `cme-demo.yaml` 檔案，並在稍後串流播放轉碼後的影片。

更新邊緣快取服務

此時建議更新 Edge Cache Service 設定，以便稍後使用服務的 IP 串流影片。邊緣快取服務 YAML 檔案可讓我們列出邊緣快取服務將接受要求的全部主機名稱/IP。此時我們只將 `demo.cme.com` 指定為主機。如要為這個網域提供名稱解析服務，您可以設定 DNS 區域。不過，更簡單的解決方法是將 IP 位址新增至 `yaml` 檔案中的主機清單。再次編輯 YAML 檔案，將其修改為下列內容：

注意：在 `hosts` 下方，將 `IPADDRESS` 替換為先前輸出內容中的 `ipv4Addresses`

```
name: cme-demo
routing:
  hostRules:
    - hosts:
        - demo.cme.com
        - IPADDRESS
      pathMatcher: routes
  pathMatchers:
    - name: routes
      routeRules:
        - headerAction:
            responseHeadersToAdd:
              - headerName: x-cache-status
                headerValue: "{cdn_cache_status}"
          matchRules:
            - prefixMatch: /
          origin: cme-origin
          priority: 100
          routeAction:
            cdnPolicy:
              cacheKeyPolicy: {}
              cacheMode: FORCE_CACHE_ALL
              defaultTtl: 3600s
              signedRequestMode: DISABLED
        - headerAction:
            responseHeadersToAdd:
              - headerName: x-cache-status
                headerValue: "{cdn_cache_status}"
          matchRules:
            - pathTemplateMatch: /**.m3u8
          origin: cme-origin
          priority: 25
          routeAction:
            cdnPolicy:
              cacheKeyPolicy: {}
              cacheMode: FORCE_CACHE_ALL
              defaultTtl: 1s
              signedRequestMode: DISABLED
        - headerAction: {}
          matchRules:
            - pathTemplateMatch: /**.ts
          origin: cme-origin
          priority: 50
          routeAction:
            cdnPolicy:
              cacheKeyPolicy: {}
              cacheMode: FORCE_CACHE_ALL
```

```
defaultTtl: 2s
signedRequestMode: DISABLED
```

如要反映變更，我們只需要重新匯入 YAML 檔案。在 Cloud Shell 終端機執行下列指令：

指令

```
gcloud edge-cache services import cme-demo \
  --source=cme-demo.yaml
```

檢查指令的輸出內容，確認 IP 位址是否顯示在主機清單中。

此時，Edge Cache Service 執行個體會接受以 `"demo.cme.com"` 或 IP 位址做為主機的要求。

12. 產生輸入信號

我們已設定所有必要服務，現在要生成直播輸入訊號。在本節中，我們將執行下列操作：

1. 安裝免費的開放原始碼軟體 FFmpeg
2. 將測試直播訊號傳送至輸入/頻道

1. 安裝 FFmpeg

FFmpeg 是免費的開放原始碼軟體專案，內含一系列程式庫和程式，可處理影片、音訊和其他多媒體檔案和串流。在 Cloud Shell 終端機中，使用下列指令安裝 FFmpeg：

指令

```
sudo apt install ffmpeg -y
```

安裝完成後，請檢查 FFmpeg 版本，確認是否已正確安裝：

注意：上述指令只會在 Cloud Shell 執行個體生命週期內安裝 FFmpeg。如果使用者離開 Cloud Shell 執行個體或開啟新的 Cloud Shell，則需要重新安裝 `FFmpeg`。

指令

```
ffmpeg -version
```

輸出內容範例

```
ffmpeg version 4.3.4-0+deb11u1 Copyright (c) 2000-2021 the FFmpeg developers
built with gcc 10 (Debian 10.2.1-6)
...
```

FFmpeg 已正確安裝。

2. 啟動直播信號至輸入/頻道

FFmpeg 安裝完成後，我們會將測試輸入串流傳送至輸入端點，以生成直播串流。

在 Cloud Shell 終端機執行下列指令，使用我們在「建立及設定輸入端點」一節中建立的 URI 環境變數。

指令

```
ffmpeg -re -f lavfi -i "testsrc=size=1280x720 [out0]; sine=frequency=500 [out1]" \
-acodec aac -vcodec h264 -f flv $URI
```

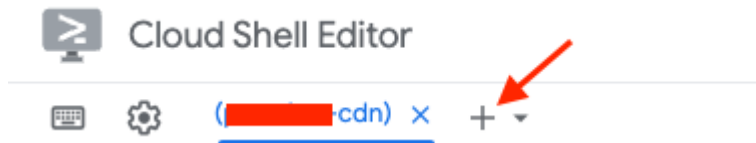
您應該會看到 FFmpeg 傳送測試即時信號。指令不會傳回提示。系統會持續產生信號，直到你停止為止。在實驗室的其餘部分，您需要開啟新的 Cloud Shell 視窗。

13. 開啟新的 Cloud Shell

匯入：請勿略過這個步驟。我們需要新的 Cloud Shell 才能完成實驗室

此時您需要開啟新的 Cloud Shell 視窗，才能繼續進行實驗室操作，因為 FFmpeg 會持續執行，直到您按下 <CTRL+C> 停止指令，並停止產生即時信號為止。

按一下目前 Cloud Shell 終端機名稱旁的「+」號。系統會開啟額外的 Cloud Shell 視窗。



在剛開啟的 Cloud Shell 視窗中，完成實驗室的其餘部分。

設定環境變數

由於這是新的 Cloud Shell，我們需要再次設定環境變數。我們將使用 `source` 指令設定環境變數。

指令

```
source ~/env_variables.txt
```

注意：上述指令只會為 Cloud Shell 執行個體生命週期設定環境變數。如果使用者離開 Cloud Shell 執行個體或開啟新的 Cloud Shell，則必須重新執行 `source` 指令。

確認已設定變數

請確認所有必要環境變數都已設定。輸出內容應會顯示總共 6 個環境變數。

指令

```
env | grep -E  
'DEVSHELL_PROJECT_ID=|LOGNAME|PROJECT_NUMBER|LOCATION|INPUT_ID|CHANNEL_ID'
```

輸出內容範例

```
LOCATION=us-west2  
DEVSHELL_PROJECT_ID=<YOUR_PROJECT_ID>  
LOGNAME=<YOUR_USERNAME>  
PROJECT_NUMBER=<YOUR_PROJECT_NUMBER>  
INPUT_ID=lab-live-input  
CHANNEL_ID=lab-live-channel
```

14. 確認直播訊號是否正在轉碼

我們會執行 `curl` 來描述管道。輸出內容應會顯示 `streamingState` 從 `"AWAITING_INPUT"` 變更為 `"STREAMING"`

指令

```
curl -s -X GET -H "Authorization: Bearer "$(gcloud auth application-default print-access-token)
"https://livestream.googleapis.com/v1/projects/$PROJECT_NUMBER/locations/$LOCATION/channels/$CHANNEL_ID" | grep "streamingState"
```

在輸出 JSON 檔案回應中，您應該會看到 `"streamingState": "STREAMING"`，表示頻道正在串流，且即時訊號正在轉碼。

我們也來驗證值區的內容，其中應該會顯示資訊清單檔案和幾個 TS 影片片段。在 Cloud Shell 中執行下列指令，列出本實驗室開頭建立的 bucket 內容，以及 Live Stream API 用來輸出轉碼後即時信號資訊清單和 TS 影片片段的內容：

指令

```
gcloud storage ls --recursive gs://live-streaming-storage-$LOGNAME/**
```

輸出內容範例

```
gs://live-streaming-storage-$LOGNAME/
gs://live-streaming-storage-$LOGNAME/main.m3u8
gs://live-streaming-storage-$LOGNAME/mux_video_ts/index-1.m3u8
gs://live-streaming-storage-$LOGNAME/mux_video_ts/segment-0000000016.ts
gs://live-streaming-storage-$LOGNAME/mux_video_ts/segment-0000000017.ts
gs://live-streaming-storage-$LOGNAME/mux_video_ts/segment-0000000018.ts
gs://live-streaming-storage-$LOGNAME/mux_video_ts/segment-0000000019.ts
gs://live-streaming-storage-$LOGNAME/mux_video_ts/segment-0000000020.ts
gs://live-streaming-storage-$LOGNAME/mux_video_ts/segment-0000000021.ts
gs://live-streaming-storage-$LOGNAME/mux_video_ts/segment-0000000022.ts
...
```

如下所示：

- HTTP 即時串流資訊清單檔案：`main.m3u8`
- 對應的 TS 影片片段：一系列編號檔案 `segment-000000000X.ts`

此時，我們已完成下列作業：

- Live Streaming API：透過 Live Streaming API 產生直播信號並轉碼至 bucket
- Media CDN：設定 Media CDN，並將 Live Streaming 儲存空間值區設為 Media CDN 的來源。

在接下來的章節中，我們會驗證 Edge Cache Service，然後使用 Media CDN 任播 IP 位址串流轉碼後的影片。

15. 確認 Edge Cache Service 執行個體是否正常運作

在本節中，我們會驗證 Edge Cache Service 執行個體是否正常運作。為此，我們將使用 Edge Cache Service 服務的 IP 位址，嘗試從 Edge Cache Service 執行個體存取檔案。首次存取物件時，系統尚未快取該物件。我們應該會觀察到快取 `MISS`。對於第一個要求，系統會從來源讀取物件，並在邊緣快取。由於物件現在已快取在邊緣，因此後續所有存取相同檔案的嘗試都會傳回快取 `HIT`。請驗證這項行為：

在 Cloud Shell 中執行下列 `curl` 指令，存取儲存在邊緣快取來源中的轉碼影片資訊清單檔案：

指令

```
curl -svo /dev/null --resolve demo.cme.com:80:<Replace_With_Edge_Cache_IP> \
"http://demo.cme.com/main.m3u8"
```

請注意，我們使用 Edge Cache Service 執行個體的 IP 位址來解析其名稱。請務必使用 `demo.cme.com:<IP>`，其中 IP 是我們剛才建立的 Edge Cache Service 執行個體 IP。

在輸出內容中尋找 `x-cache-status` 標題。

輸出內容範例

```

Added demo.cme.com:80:34.104.35.152 to DNS cache
* Hostname demo.cme.com was found in DNS cache
*   Trying 34.104.35.152:80...
* Connected to demo.cme.com (34.104.35.152) port 80 (#0)
> GET /main.m3u8 HTTP/1.1
> Host: demo.cme.com
> User-Agent: curl/7.74.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
< x-guploader-uploadid: ADPycdtKtflWt4Kha5YxXNNRwO-Eu6fGSPs-T-XY4HJmNM046VJyWlD4EAK-8a6SegxjWq3o1gTPqZbpkU_sjW__HPAdDw
< date: Wed, 07 Dec 2022 18:23:46 GMT
< last-modified: Wed, 07 Dec 2022 18:23:45 GMT
< etag: "6bff620ccca4a9849ba4e17fa7c521fb"
< x-goog-generation: 1670437425805400
< x-goog-metageneration: 1
< x-goog-stored-content-encoding: identity
< x-goog-stored-content-length: 193
< content-type: application/x-mpegURL
< x-goog-hash: crc32c=sP03zw==
< x-goog-hash: md5=a/9iDMykqYSbpOF/p8Uh+w==
< x-goog-storage-class: STANDARD
< accept-ranges: bytes
< content-length: 193
< server: Google-Edge-Cache
< x-request-id: fd25285b-fc1a-4fd4-981a-c50ead2c85ed
< x-xss-protection: 0
< x-frame-options: SAMEORIGIN
< x-cache-status: den;miss
< cache-control: public,max-age=3600
<
{ [193 bytes data]
* Connection #0 to host demo.cme.com left intact

```

請注意，由於物件尚未快取，且是從來源讀取，因此會發生快取**未命中**的情況。

現在我們會對 `m3u8` 檔案提出多項要求，如果一切設定正確，Media CDN 應該會開始從快取提供內容。下列指令會發出 10 項 curl 要求，並只列印 `x-cache-status` 標頭。

指令

```

for i in {1..10};do curl -Is --resolve demo.cme.com:80:<Replace_With_Edge_Cache_IP>
"http://demo.cme.com/main.m3u8" | grep x-cache-status;done

```

輸出內容應包含快取 `hit` 和 `miss`。如果輸出內容顯示快取命中，表示 Media CDN 運作正常。

輸出內容範例

```
x-cache-status: den;miss
x-cache-status: den;hit
x-cache-status: den;hit
x-cache-status: den;hit
x-cache-status: den;hit
x-cache-status: den;hit
x-cache-status: den;hit
x-cache-status: den;hit
x-cache-status: den;hit
x-cache-status: den;hit
```

請注意，由於物件現在已快取在邊緣，因此快取會**命中**。Cloud Media Edge Service 運作正常。

16. 使用 VLC 串流播放轉碼後的即時訊號影片

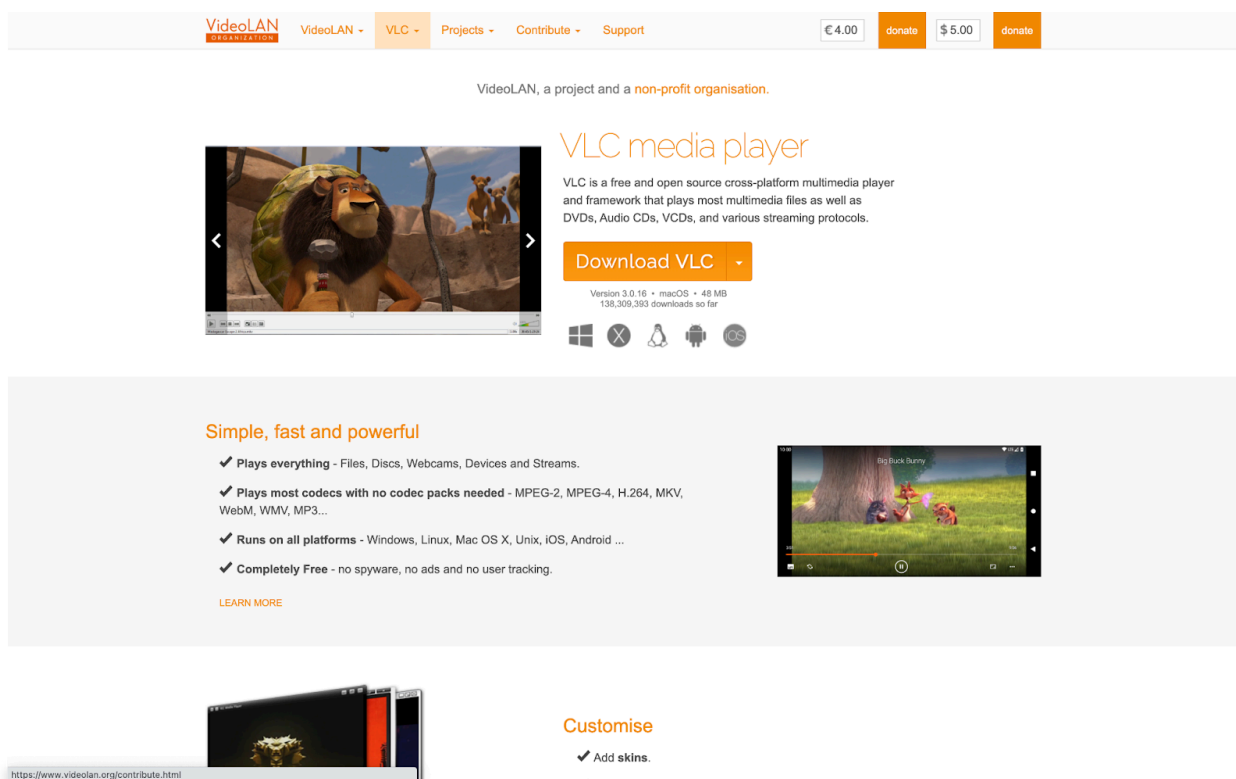
現在要將所有步驟連結起來，也就是將我們目前為止所做的所有工作整合在一起：

- 我們建立了一個名為 `live-streaming-storage-$LOGNAME` 的值區，用來接收由 Live Streaming API 轉碼為 HTTP 即時串流內容的直播信號結果。
- 我們設定了 Live Streaming API。
- 我們使用 FFmpeg 啟動 RTMP 直播信號，將內容提供給 Live Streaming API 輸入/頻道。
- 我們已確認直播信號已傳送至頻道，且頻道處於 `streaming` 模式。
- 我們已確認系統生成轉碼後的檔案 (資訊清單 + TS 片段)，並儲存在 `live-streaming-storage-$LOGNAME` 值區中。
- 系統已設定名為 `cme-origin` 的邊緣快取來源，並以 GCS bucket `live-streaming-storage-$LOGNAME` 做為來源。
- 已設定名為 `cme-demo` 的 Edge Cache 執行個體，並以 `cme-origin` 做為來源。
- 我們已驗證 Edge Cache 服務執行個體的行為 (快取失敗、快取命中)。

現在我們可以使用影片播放器，透過 Media CDN 快取串流播放轉碼後的即時訊號。為此，我們將使用 [VLC Player](#)。VLC Player 是免費的開放原始碼跨平台多媒體播放器和架構，可播放大多數多媒體檔案。可播放自動調整媒體格式 (例如 DASH 和 HLS)。這項功能採用 [自動調整串流](#) 原則，播放器會根據網路連線品質和可用頻寬，自動調整播放影片的畫質。在我們剛完成的轉碼作業中，我們使用預設預設集，並「僅」生成兩種品質：SD 和 HD。開始在播放器中播放影片時，如果網路連線品質良好，影片應該會先以 SD 格式播放，然後快速切換為 HD 格式。

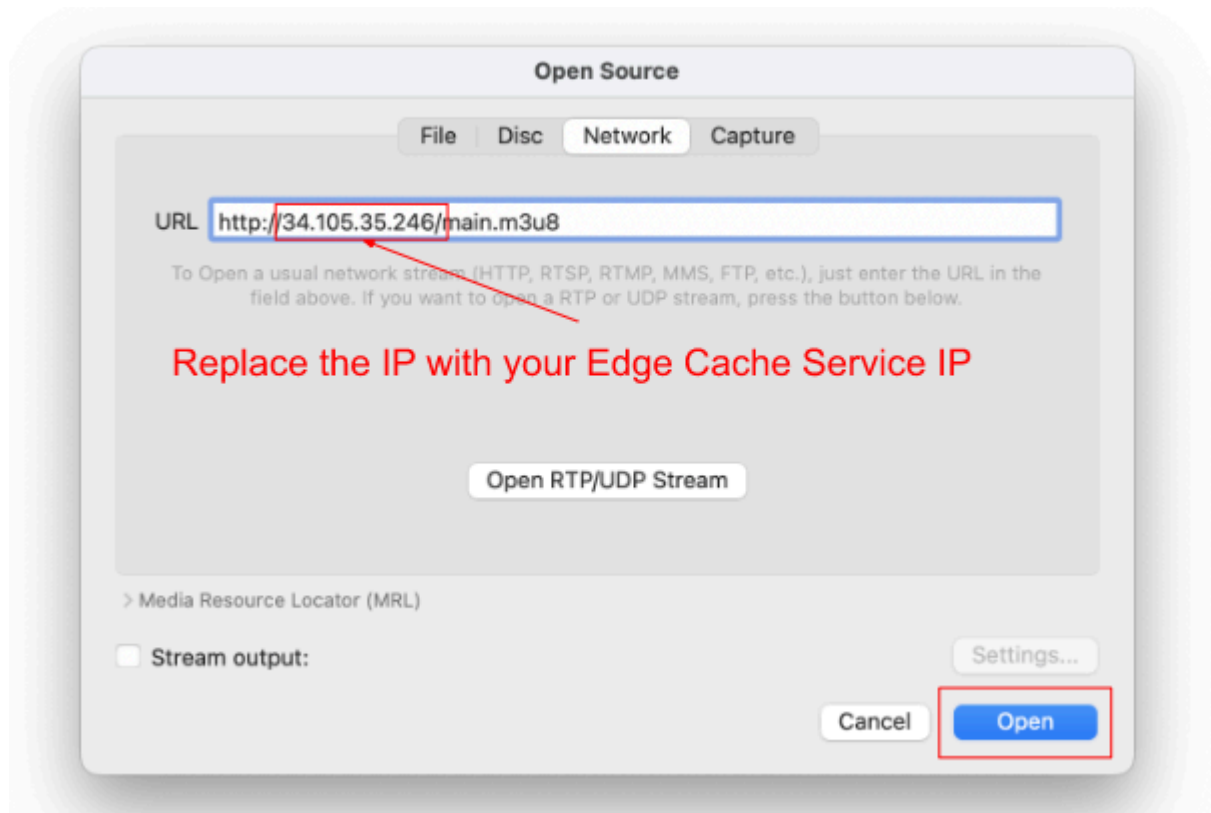
我們會串流播放經過轉碼的 HTTP 即時串流 (廣泛支援的 Apple 影片格式) 直播信號。對應檔案稱為 `main.m3u8`，也就是 HLS 資訊清單。資訊清單會指向 TS 影片片段。

如要使用 VLC Player，請前往 <https://www.videolan.org/vlc/>，然後下載適用於筆電作業系統的播放器版本。VLC 支援 Windows、MacOSX、Linux、Android 和 iOS。



在筆記型電腦上安裝 Player 並啟動。在接下來的幾個步驟中，我們將使用 MacOSX 版本的播放器。

如要播放影片，請依序前往「檔案」/「開啟網路」：

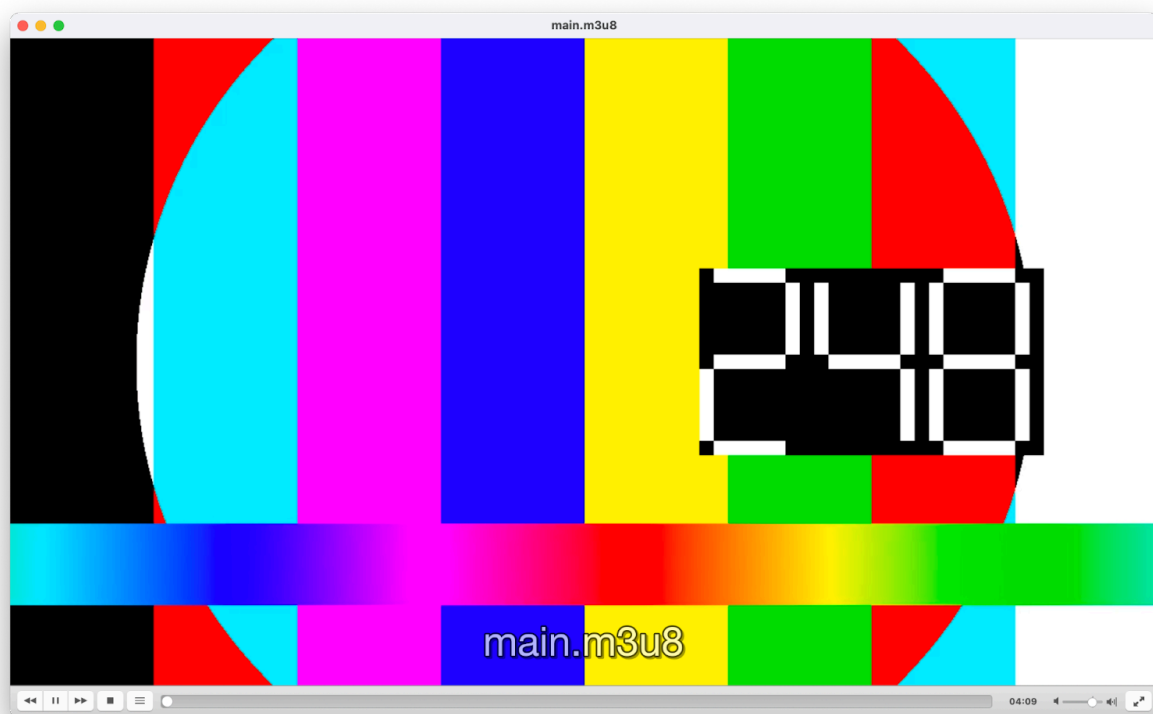


使用下列設定：

- 網址：http://<Replace_With_Edge_Cache_IP>/main.m3u8。這是我們要串流播放的影片網址。通知：
- Media CDN 執行個體的 IP：`34.105.35.246`。請替換為您部署的 Cloud Media Service IP。
- 資訊清單影片檔案的路徑：「/」。這是我們在 `live-streaming-storage-$LOGNAME` bucket 中儲存轉碼直播訊號檔案時使用的路徑。路徑為根路徑「/」。
- 資訊清單影片檔案的名稱：HLS 資訊清單檔案，`main.m3u8`。

然後按一下「開啟」。這時應該會看到轉碼後的即時影像開始播放。影片會如下方螢幕截圖所示。畫面上的計數器會以 1 為增量遞增，你應該會聽到連續的嗶聲。

這是由 FFmpeg 產生的基本 RTMP 測試直播信號，經由 Live Streaming API 轉碼為 HTTP 即時串流，並透過 Media CDN 快取提供服務：



如要使用其他 HLS 和 MPEG DASH 播放器，請考慮下列選項：

- Quicktime Player - Mac 預設安裝的播放器。同樣地，開啟與 <http://34.104.36.157/main.m3u8> 的網路連線，並將 IP 位址替換為 Edge Cache Service 執行個體的 IP 位址。
-

步驟 17 · 約 10 分鐘

17. 監控 Media CDN

SME 團隊已建立 Media CDN 資訊主頁範本 - <https://gist.github.com/elithrar/1c511d00f5cd3736fb2a3897867209c1>。

如要安裝，請在 Cloud Shell 視窗中執行下列指令：

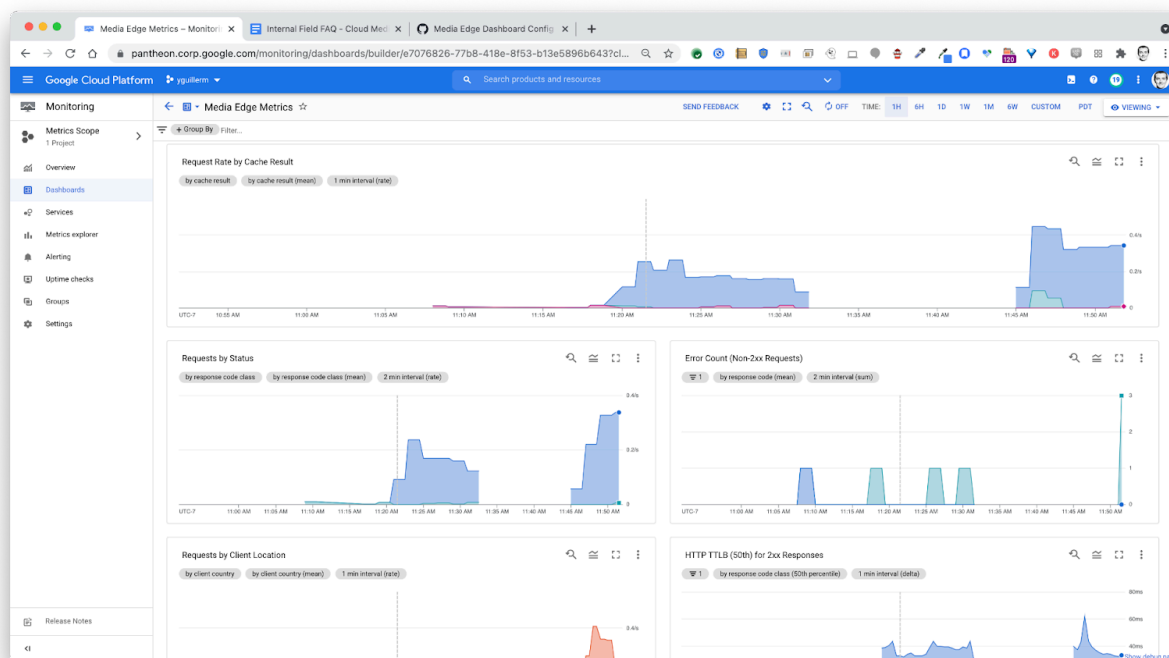
下載 YAML 檔案：

```
curl
https://gist.githubusercontent.com/elithrar/1c511d00f5cd3736fb2a3897867209c1/raw/3cb7
0855304f29e5c06b8d63063196354db0dec3/media-edge-20210208-dashboard --output media-
edge-20210208-dashboard.yaml
```

建立 Cloud Monitoring 資訊主頁：

```
gcloud monitoring dashboards create --config-from-file media-edge-20210208-
dashboard.yaml
```

設定程序可能需要幾分鐘。前往 Google Cloud 控制台，依序點選 3 條橫線 > 「作業」 > 「監控」 > 「資訊主頁」。您應該會看到名為「Media Edge Metrics」的資訊主頁。點選該指標，即可查看下列指標：



步驟 18 · 約 10 分鐘

18. 清理實驗室環境

恭喜您完成本實驗室！在本節中，我們將刪除整個實驗室建立的所有資源。

停止 FFmpeg 信號：

在執行 FFmpeg 的 Cloud Shell 終端機中按下 `<CTRL+C>`。

停止直播頻道：

指令

```
curl -X POST \  
-H "Authorization: Bearer "$(gcloud auth application-default print-access-token) \  
-H "Content-Type: application/json; charset=utf-8" \  
-d "" \  
"https://livestream.googleapis.com/v1/projects/$PROJECT_NUMBER/locations/$LOCATION/channels/$CHANNEL_ID:stop"
```

刪除直播頻道：

指令

```
curl -X DELETE -H "Authorization: Bearer "$(gcloud auth application-default print-access-token) \  
"https://livestream.googleapis.com/v1/projects/$PROJECT_NUMBER/locations/$LOCATION/channels/$CHANNEL_ID"
```

刪除 Live Streaming Input Endpoint：

指令

```
curl -X DELETE \  
-H "Authorization: Bearer "$(gcloud auth application-default print-access-token) \  
"https://livestream.googleapis.com/v1/projects/$PROJECT_NUMBER/locations/$LOCATION/inputs/$INPUT_ID"
```

刪除 GCS bucket：

指令

```
gsutil rm -r gs://live-streaming-storage-$LOGNAME
```

刪除邊緣快取服務執行個體：

指令

```
gcloud edge-cache services delete cme-demo
```

系統提示時輸入「Y」，確認要刪除

刪除邊緣快取來源：

指令

```
gcloud edge-cache origins delete cme-origin
```

系統提示時輸入「Y」，確認要刪除

刪除自訂資訊主頁

指令

```
gcloud monitoring dashboards delete $(gcloud monitoring dashboards list --  
filter="displayName:Media Edge Metrics" --format="value(name)")
```
